

Adaptive Personalized Learning Recommendation Using Reinforcement Learning: A Comparative Analysis of Q-Learning and Monte Carlo Control

Shamazda Islam

sec or id

Ramisa Sharar Nidhi

sec or id

Abstract

Online learning platforms offer diverse educational resources, but students differ in pace, prior knowledge, and performance, making fixed activity sequences suboptimal. Personalized learning recommendation addresses this by selecting activities based on a learner’s current state, and Reinforcement Learning (RL) is well suited to this task as a sequential decision-making problem.

This work develops an adaptive personalized learning recommendation framework using student interaction data from the EdNet-KT3 dataset [1]. Learning states are constructed from performance-related and behavioral features, including mastery, recent accuracy, weak knowledge areas, and activity history. The recommendation problem is formulated as an off-line RL environment with three observable actions—QUIZ, PRACTICE, and REVISION—using an empirical transition model with action masking to restrict agents to actions supported by observed data. Two model-free algorithms, Q-Learning and Monte Carlo Control, are trained and compared against a random-policy baseline using average return, mastery success rate, behavioral agreement, convergence behavior, computational time, and multi-seed evaluation.

The results provide a transparent comparison of two fundamental RL approaches for learning adaptive recommendation policies from large-scale historical educational interaction data.

Keywords— Personalized Learning, Reinforcement Learning, Q-Learning, Monte Carlo Control, Adaptive Learning, Educational Recommendation, EdNet-KT3.

1 Introduction

Online learning systems have become an important component of modern education by providing students with access to large collections of digital educational resources. However, students do not learn in the same way or at the same pace. They have different levels of prior knowledge, learning abilities, strengths, and weaknesses. A learning activity that is appropriate for one student may not be

suitable for another student at the same stage. Traditional e-learning systems often provide predefined learning sequences or static recommendations that may not adapt adequately to changes in student performance. This creates a need for adaptive personalized learning systems that can continuously observe a student’s learning condition and recommend appropriate activities. Personalized educational recommendation is particularly challenging because it is a sequential decision-making problem: the activity selected at the current stage can influence future student knowledge and subsequent learning requirements. Reinforcement Learning provides a suitable framework for solving this problem by enabling an agent to learn a policy that selects actions according to the current state while maximizing long-term cumulative reward [2], [3].

Previous research has investigated personalized learning, knowledge tracing, adaptive learning systems, and reinforcement learning-based recommendation. Knowledge tracing approaches have been widely used to estimate the evolving knowledge of students from sequential learning interactions, with Deep Knowledge Tracing being an influential deep learning approach in this area [4]. In personalized learning recommendation, Tang et al. formulated the problem as a sequential decision-making problem and demonstrated the usefulness of reinforcement learning for selecting learning materials based on learner states [2]. Similarly, Tan et al. proposed an adaptive learning recommendation strategy based on Deep Q-Learning, demonstrating the potential of RL for constructing individualized learning paths [5]. Chen et al. also investigated adaptive learning recommendation using learner information and learning history [6]. More broadly, recent research has shown growing interest in the application of reinforcement learning in education for adaptive tutoring, instructional sequencing, personalized feedback, and learning recommendation. A recent systematic literature review also highlighted the increasing use of RL in educational applications while identifying challenges related to evaluation, methodology, and real-world deployment [7]. Although deep reinforcement learning approaches can handle complex state spaces, they often require substantial computational resources and may be difficult to interpret. Furthermore, when historical educational data are used, not every possible action has an observed outcome for every student state. Therefore, allowing an RL agent to select unsupported actions may introduce artificial assumptions into the learning environment.

Based on these limitations, this project develops a transparent offline personalized learning recommendation framework using the EdNet-KT3 dataset [1] and compares two fundamental model-free reinforcement learning algorithms: Q-Learning and Monte Carlo Control [3]. Rather than constructing artificial outcomes for all possible educational activities, the proposed RL environment uses three directly observable question-based actions: QUIZ, PRACTICE, and REVISION. Student learning states are constructed from observed performance and behavioral information, and an empirical transition model is generated from historical interactions. In addition, action masking is applied to restrict each RL agent to state-action combinations supported by the training data. This reduces unsupported transitions and improves the reliability of the offline RL environment. The project provides a controlled comparison between Q-Learning

and Monte Carlo Control under the same dataset, state representation, reward structure, transition model, and evaluation framework. The study therefore contributes a transparent and computationally efficient approach for investigating how classical reinforcement learning algorithms can support adaptive educational recommendation.

1.1 Our Contribution

The main contributions of this project are summarized as follows:

1. **Adaptive RL-Based Personalized Learning Framework:** We developed an adaptive personalized learning recommendation framework using reinforcement learning and historical student interaction data from the EdNet-KT3 dataset [1].
2. **Student Learning State Construction:** Student learning states are constructed using observed performance and behavioral information, including knowledge mastery, recent accuracy, weak knowledge areas, and learning activity history.
3. **Observable Educational Action Space:** The proposed system uses three observable educational actions:
 - **Quiz**
 - **Practice**
 - **Revision**These actions are constructed from observable question-level student interactions rather than assigning artificial outcomes to unsupported activities.
4. **Empirical Offline Transition Model:** The RL environment is constructed from empirical student transitions observed in the dataset. This enables the agents to learn from historical student behavior while reducing reliance on artificially generated transitions.
5. **Action Masking for Supported Actions:** Action masking is incorporated to restrict RL agents to actions empirically supported by the training data for a particular student state. This improves the reliability of the offline learning environment.
6. **Comparison of Two Model-Free RL Algorithms:** Two classical reinforcement learning algorithms are implemented and compared:
 - **Q-Learning**
 - **Monte Carlo Control**

Both algorithms are evaluated under the same experimental environment and conditions [3].

7. **Baseline Comparison:** A random policy is included as a baseline to evaluate whether the learned reinforcement learning policies provide improvements over non-adaptive action selection.
8. **Comprehensive Evaluation:** The proposed algorithms are evaluated using multiple criteria:
 - **Average Return**
 - **Return Standard Deviation**
 - **Mastery Success Rate**
 - **Behavioral Agreement**
 - **Average Episode Steps**
 - **Learning Convergence**
 - **Computational Time**
 - **Multi-Seed Performance**

2 Materials and Methods

The proposed system formulates personalized learning recommendation as a *Reinforcement Learning (RL)* problem. Student interaction histories from the EdNet-KT3 dataset were processed to construct sequential learning trajectories. Each trajectory was converted into RL transitions consisting of:

$$(s_t, a_t, r_t, s_{t+1}) \tag{1}$$

where:

- s_t represents the student’s learning state before an activity,
- a_t represents the observed learning activity,
- r_t represents the reward associated with the learning outcome, and
- s_{t+1} represents the student’s learning state before the subsequent activity.

The system was designed to avoid information leakage. Rather than constructing a state after observing the outcome of an activity, the state was generated using only the student’s learning history **before the current bundle began**. The outcome of the bundle was subsequently used to calculate the reward, while the state before the following bundle became the next state. This bundle-level design provides a cleaner representation of the sequential learning process.

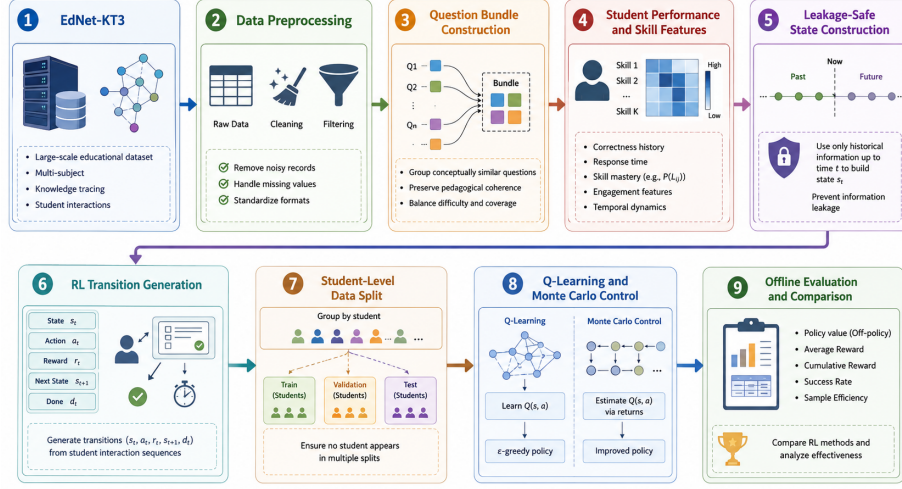


Figure 1: Overview of the proposed RL-based knowledge tracing pipeline, from raw EdNet-KT3 data to offline policy evaluation.

2.1 Dataset Description

2.1.1 EdNet Dataset

This study used the **EdNet educational dataset**, a large-scale dataset of student interactions collected from the Santa AI-based self-learning platform. EdNet contains diverse educational activities and was specifically designed to support research in areas such as knowledge tracing, student modeling, and learning path recommendation. The complete EdNet dataset contains over 131 million interactions from more than 784,000 students and organizes student activity data into four hierarchical versions: KT1, KT2, KT3, and KT4[1].

The dataset and its documentation are available from the official EdNet repository.¹

In this project, **EdNet-KT3** was selected because it contains richer learning interactions than question-only datasets. In addition to question-solving activities, KT3 records activities related to lecture consumption and explanation viewing. This makes KT3 particularly suitable for modeling student learning behavior and constructing an adaptive learning recommendation environment.

For computational feasibility, **100 student interaction files** were selected for the experiment. Each student’s interactions were stored separately and then combined into a unified dataset for preprocessing.

2.1.2 Student Interaction Data

The primary student interaction records contain the following variables.

¹<https://github.com/riid/ednet>

Table 1: Student Interaction Features

Feature	Description
<code>timestamp</code>	Time at which the interaction occurred
<code>action_type</code>	Type of interaction, such as enter, respond, submit, or quit
<code>item_id</code>	Identifier of the educational item
<code>source</code>	Source or learning context of the activity
<code>user_answer</code>	Student’s selected answer
<code>platform</code>	Platform used by the student
<code>user_id</code>	Unique student identifier
<code>item_type</code>	Category of educational item

The `item_id` field was particularly important for identifying different learning activities. The prefix of the identifier was used to distinguish item types:

- `b` → Question bundle
- `q` → Individual question
- `l` → Lecture
- `e` → Explanation

For example, a typical question-solving sequence may contain:

Enter Bundle → Respond to Question → Submit Bundle (2)

The student interaction data were sorted chronologically for each student before feature extraction and RL transition construction.

2.1.3 Question Metadata

The question metadata file contains information associated with individual questions.

Table 2: Question Metadata Features

Feature	Description
<code>question_id</code>	Unique question identifier
<code>bundle_id</code>	Identifier of the question bundle
<code>explanation_id</code>	Associated explanation identifier
<code>correct_answer</code>	Correct answer option
<code>part</code>	Educational content section
<code>tags</code>	Associated skill or knowledge tags
<code>deployed_at</code>	Question deployment timestamp

The `correct_answer` variable was compared with the student’s submitted answer to determine whether a response was correct:

$$is_correct = \begin{cases} 1, & \text{if user answer} = \text{correct answer} \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

The **tags** variable was used to associate question attempts with knowledge skills. These skills were subsequently used to calculate student mastery-related features.

2.1.4 Lecture Metadata

The lecture metadata contains the following variables.

Table 3: Lecture Metadata Features	
Feature	Description
lecture_id	Unique lecture identifier
part	Educational content section
tags	Knowledge or skill tag
video_length	Duration of the lecture in milliseconds
deployed_at	Lecture deployment timestamp

Some lecture records contain:

$$video_length = -1 \quad (4)$$

which indicates that valid video duration information was unavailable. Therefore, such values were treated as missing rather than valid lecture durations.

For lectures with valid durations, the video length was converted from milliseconds to seconds:

$$VideoLength_{seconds} = \frac{VideoLength_{milliseconds}}{1000} \quad (5)$$

Lecture sessions were extracted from student interaction sequences using the time difference between lecture **enter** and **quit** events.

2.1.5 Question Attempt Construction

Question attempts were reconstructed from the raw interaction logs. When a student entered a bundle, responded to one or more questions, and submitted the bundle, the corresponding responses were grouped into a question-bundle attempt.

For each attempt, the following information was retained:

- student identifier,
- bundle identifier,
- question identifier,

- timestamp,
- submitted answer,
- correctness,
- source, and
- platform.

Bundle-level accuracy was calculated as:

$$Accuracy_{bundle} = \frac{\text{Number of Correct Answers}}{\text{Number of Valid Answers}} \quad (6)$$

Using bundle-level outcomes was important because EdNet questions can occur as groups sharing common instructional material.

2.1.6 Skill Mastery and Recent Performance

Student performance was represented using both overall and recent learning history.

For each skill, mastery was estimated as:

$$Mastery_{u,k} = \frac{CorrectAnswers_{u,k}}{Attempts_{u,k}} \quad (7)$$

where u represents a student and k represents a knowledge skill.

To capture changes in recent performance, a rolling window of the previous **10 interactions** was used:

$$RecentAccuracy_t = \frac{1}{N} \sum_{i=t-N}^{t-1} Correct_i \quad (8)$$

where $N = 10$.

The current question outcome was excluded from its own state representation. This ensured that the RL agent could not access future outcome information when making a decision. The implementation therefore uses a leakage-safe recent performance calculation based on shifted historical outcomes.

2.2 Tools, Models, and Algorithms Used

2.2.1 Software and Computational Tools

The complete experimental pipeline was implemented in **Python** using the Kaggle notebook environment. The main libraries included:

The implementation was designed to run directly on Kaggle using paths under `/kaggle/input/` for datasets and `/kaggle/working/` for generated outputs.

Table 4: Lecture Metadata Features

Feature	Description
lecture_id	Unique lecture identifier
part	Educational content section
tags	Knowledge or skill tag
video_length	Duration of the lecture in milliseconds
deployed_at	Lecture deployment timestamp

2.2.2 Reinforcement Learning Formulation

The personalized learning problem was formulated as a Markov Decision Process (MDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma) \quad (9)$$

where:

- $\mathcal{S}$ is the set of student learning states,
- $\mathcal{A}$ is the action space,
- $\mathcal{P}$ represents state transitions,
- $\mathcal{R}$ is the reward function, and
- γ is the discount factor.

The RL agent observes the student’s current learning state and selects an appropriate learning activity.

2.2.3 State Representation

The student’s state was constructed using learning information available **before the current question bundle**. The state incorporated:

1. Recent learning accuracy,
2. Recent skill mastery,
3. Learning history,
4. Educational activity progression.

Continuous performance features were discretized into three levels. For mastery and accuracy:

$$Level(x) = \begin{cases} 0, & x < 0.40 \\ 1, & 0.40 \leq x < 0.70 \\ 2, & x \geq 0.70 \end{cases} \quad (10)$$

Therefore:

- Level 0 represents low performance,
- Level 1 represents medium performance,
- Level 2 represents high performance.

The state was constructed at the **bundle level**, rather than independently for every question. This prevents the possibility of attaching a state generated from one question to the outcome of another question within the same bundle.

2.2.4 Action Space

The conceptual learning activity space consisted of four actions:

$$\mathcal{A} = \{LESSON, QUIZ, PRACTICE, REVISION\} \quad (11)$$

The actions were encoded as shown in Table ??.

Table 5: Conceptual Learning Actions

Action	Action ID
LESSON	0
QUIZ	1
PRACTICE	2
REVISION	3

However, an important methodological decision was made in the final implementation. The primary RL experiment used directly observable question-based actions because these actions have directly associated question outcomes. Lecture activities and explanations were retained for descriptive feature extraction but were not artificially assigned question outcomes.

The primary observable action categories were therefore derived from the EdNet **source** field:

- diagnostic activities $\rightarrow$ QUIZ,
- sprint activities $\rightarrow$ PRACTICE,
- review activities $\rightarrow$ REVISION.

This design avoids artificially creating counterfactual outcomes for lecture recommendations. Because no observed source in the processed data mapped to LESSON, the RL environment used in every experiment reported in this paper contains only three actions, $\mathcal{A} = \{QUIZ, PRACTICE, REVISION\}$, with contiguous action ids 0–2; the four-action encoding in Table 5 is the conceptual action space only and was not used for training or evaluation.

2.2.5 Reward Function

The reward function was designed to represent learning performance and improvement after an observed activity. In general:

- positive outcomes receive positive rewards,
- poor learning outcomes receive lower or negative rewards,
- improvements in learning performance contribute positively to the reward.

The reward is associated with the outcome of a bundle after the pre-action state has been constructed. Thus, the RL transition follows:

$$(s_t, a_t, r_t, s_{t+1}) \quad (12)$$

The state s_t is generated from historical information, the observed learning activity becomes a_t , the bundle outcome contributes to r_t , and the pre-action state of the next bundle becomes s_{t+1} . This structure prevents target leakage because future outcomes are not included in the current state.

2.2.6 Q-Learning

Q-Learning was selected as the first RL algorithm because it is a model-free, off-policy algorithm that learns the optimal action-value function.

The Q-value update is:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right] \quad (13)$$

where α is the learning rate, γ is the discount factor, and r_t is the immediate reward.

The learned policy is:

$$\pi(s) = \arg \max_a Q(s, a) \quad (14)$$

Hyperparameters

The main configuration used in the final implementation is shown in Table ??.

These parameters were selected to provide sufficient exploration during early training while gradually encouraging exploitation of learned Q-values.

2.2.7 Monte Carlo Control

Monte Carlo Control was used as the second RL algorithm for comparison. Unlike Q-Learning, Monte Carlo methods estimate action values using the complete return observed from an episode.

The discounted return from time t is:

Table 6: RL Hyperparameter Configuration

Parameter	Value
Random state	42
Number of students	100
Recent performance window	10
Minimum episode length	5
Discount factor (γ)	0.95
Learning rate (α)	0.10
Training episodes	3000
Maximum steps	30
Initial epsilon	1.0
Final epsilon	0.05
Epsilon decay	0.995
Number of seeds	5

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1} \quad (15)$$

In the implemented estimator, the action-value function is updated using a running sample average of the observed first-visit returns for each state-action pair,

$$Q(s, a) \leftarrow Q(s, a) + \alpha [G_t - Q(s, a)] \quad (16)$$

where $N(s, a)$ is the number of times (s, a) has been visited so far. This is the standard first-visit Monte Carlo control update, and it does not use the fixed learning rate α used by Q-Learning; the incremental step size $1/N(s, a)$ decreases automatically as a state-action pair is visited more often. Consequently, the hyperparameter analysis in Section 3.6 for Monte Carlo Control varies only γ and the number of training episodes – the α column reported for Monte Carlo Control in Table 11 has no effect on its results and is included only to mirror the Q-Learning configuration grid.

Monte Carlo Control is useful for comparison because it learns from complete episodes rather than bootstrapping from the estimated value of the next state. Thus, the comparison investigates two different RL learning mechanisms:

- **Q-Learning:** temporal-difference bootstrapping,
- **Monte Carlo Control:** complete-episode return estimation.

2.2.8 Random Policy Baseline

A Random policy was included as a baseline. The Random policy selects an available learning action without using learned student-state values.

Its purpose is to determine whether the RL algorithms provide improvement beyond uninformed action selection. Therefore, the experimental comparison included:

1. Q-Learning,
2. Monte Carlo Control,
3. Random baseline.

2.2.9 Empirical Offline Transition Model

Because the dataset consists of historical student interactions rather than an environment that can be directly manipulated, an empirical offline transition model was constructed using the training data.

The model estimates transitions based on observed:

$$(s, a, r, s') \tag{17}$$

tuples.

Importantly, the empirical transition model was constructed using **training students only**. This prevents information from validation or test students from influencing the learned environment or policy.

2.3 Performance Evaluation

2.3.1 Student-Level Train, Validation, and Test Split

The dataset was divided at the **student level**, rather than randomly splitting individual transitions.

The eligible students were divided into:

- 70% Training,
- 15% Validation,
- 15% Testing.

The initial split allocated 70% of students to training and 30% to a temporary set, which was subsequently divided equally into validation and testing.

This design ensures that:

$$TrainStudents \cap ValidationStudents = \emptyset \tag{18}$$

$$TrainStudents \cap TestStudents = \emptyset \tag{19}$$

$$ValidationStudents \cap TestStudents = \emptyset \tag{20}$$

Therefore, no student’s interaction history appears simultaneously in training and testing. This is particularly important for personalized learning systems because splitting individual interactions randomly could allow information about the same student to appear in both training and testing.

2.3.2 Average Return

The primary RL evaluation metric was the **average discounted return**.

For an episode:

$$G = \sum_{t=0}^{T-1} \gamma^t r_t \quad (21)$$

The average return was calculated across the evaluation episodes:

$$AverageReturn = \frac{1}{N} \sum_{i=1}^N G_i \quad (22)$$

A higher average return indicates that the policy produces trajectories associated with higher cumulative learning rewards.

2.3.3 Mastery Success Rate

Mastery Success Rate was used to evaluate whether the final student states reached a desirable mastery level.

$$MasterySuccessRate = \frac{\text{Number of successful learning trajectories}}{\text{Total evaluated trajectories}} \quad (23)$$

This metric provides an outcome-oriented measure in addition to cumulative reward.

2.3.4 Behavioral Agreement

Behavioral agreement measures how frequently the action recommended by the learned RL policy matches the action observed in the historical student data.

$$BehavioralAgreement = \frac{\text{Number of matching actions}}{\text{Total evaluated actions}} \quad (24)$$

This metric should not be interpreted simply as recommendation accuracy. A learned RL policy may recommend an action different from the historical student action while still potentially being valuable. Therefore, behavioral agreement measures consistency with observed historical behavior rather than true recommendation correctness.

2.3.5 Learning Gain Analysis

Learning gain was analyzed to determine whether different learning activities were associated with improvements in student performance. Learning gain provides additional evidence about the educational effects associated with observed activities and complements the reward-based RL evaluation.

2.3.6 Convergence Analysis

The learning process was evaluated by tracking training returns across episodes. Convergence plots were generated for:

- Q-Learning,
- Monte Carlo Control.

The curves allow comparison of:

- learning stability,
- speed of improvement,
- final training performance.

A stable plateau in the learning curve indicates that the policy values have reached a relatively stable region.

2.3.7 Computational Time

The execution time required for training each RL algorithm was also measured. Computational time provides a practical comparison because an algorithm with better performance may require substantially more computational resources.

Therefore, the comparison considered:

$$Performance + Stability + ComputationalCost \quad (25)$$

2.3.8 Multi-Seed Evaluation

To reduce dependence on a single random initialization, experiments were repeated using **five different random seeds**.

The final results were summarized as:

$$Mean \pm StandardDeviation \quad (26)$$

for:

- Average Return,
- Mastery Success Rate,

- Training Return.

The multi-seed evaluation avoids relying on a single random run and improves the reliability of the experimental comparison between Q-Learning and Monte Carlo Control.

3 Experimental Analysis

3.1 Experimental Setup

The proposed reinforcement learning framework was evaluated using student interaction data extracted from the EdNet-KT3 dataset. For computational feasibility, the experiment considered interaction histories from 100 students. After preprocessing and question-attempt reconstruction, the dataset contained 51,523 question attempts, 1,547 lecture sessions, and 40,037 explanation sessions.

The question-bundle processing and leakage-safe state construction procedures subsequently produced 25,736 reinforcement learning transitions from 98 of the 100 students (two students did not meet the minimum-episode-length requirement of 5 transitions and were excluded). The final RL environment contained approximately 600 unique student learning states and three observable learning actions: $A = \{\text{QUIZ}, \text{PRACTICE}, \text{REVISION}\}$.

The experiments compared three policies: Q-Learning, Monte Carlo Control, and a Random policy baseline. The dataset was divided at the student level (68 training / 15 validation / 15 test students) to ensure that interactions from the same student did not appear simultaneously in training and evaluation sets. This prevents overly optimistic performance estimates caused by student-level information leakage.

The main hyperparameter configuration used for the experiments is shown in Table 7.

Table 7: Main experimental hyperparameter configuration.

Parameter	Value
Number of students	100
Recent performance window	10
Minimum episode length	5
Discount factor (γ)	0.95
Learning rate (α)	0.10
Training episodes	3000
Maximum steps per episode	30
Initial ϵ	1.00
Final ϵ	0.05
ϵ decay	0.995
Random seeds	5

Note: Table 8 and Table 7 report a training-episode count of 3000, matching what was used to produce every result in this paper. The use of multiple random seeds (Section 3.8) was intended to reduce dependence on a single stochastic training run and improve the reliability of the algorithmic comparison.

3.2 Overall Comparison of Reinforcement Learning Algorithms

Table 8 presents the primary comparison between Q-Learning and Monte Carlo Control, using the single-seed (random state 42) run defined by Table 7.

Table 8: Overall comparison of the reinforcement learning algorithms (single seed, random state 42).

Metric	Q-Learning	Monte Carlo
Test Average Return	30.04	28.52
Logged-Action Agreement (%)	87.88	80.85
Empirical-Best Agreement (%)	92.67	85.49
Mastery Completion Proxy (%)	49.4	46.0
Training Time (seconds)	0.50	0.37

The results indicate that Q-Learning achieved better overall evaluation performance than Monte Carlo Control under this single training seed. The average test return of Q-Learning was 30.04, compared with 28.52 for Monte Carlo Control. Q-Learning also achieved higher logged-action agreement (87.88% vs. 80.85%) and empirical-best agreement (92.67% vs. 85.49%), as well as a higher mastery completion proxy (49.4% vs. 46.0%).

Although Q-Learning achieved better learning-related evaluation results under this seed, Monte Carlo Control required less training time: approximately 0.37 seconds, compared with 0.50 seconds for Q-Learning. Therefore, the results demonstrate a trade-off between computational efficiency and policy performance.

These single-seed numbers should be read together with the multi-seed evaluation in Section 3.8, which shows that this apparent advantage for Q-Learning is not robust: averaged over five seeds, Monte Carlo Control’s test average return is marginally higher than Q-Learning’s, and the two algorithms are statistically indistinguishable on return. The single-seed comparison in this section therefore should not, on its own, be read as establishing Q-Learning as the better algorithm; see Section 3.8 for the more reliable comparison.

3.3 Comparison with the Random Baseline

A Random policy was included as a baseline to determine whether the learned RL policies performed better than uninformed action selection.

The Random baseline achieved a mastery completion proxy of approximately 42.4%. In comparison, Monte Carlo Control achieved 46.0%, while Q-Learning

achieved the highest value of 49.4%.

Table 9: Comparison with the Random baseline.

Policy	Mastery Completion Proxy (%)
Random	42.4
Monte Carlo Control	46.0
Q-Learning	49.4

The improvement of both RL algorithms over the Random baseline suggests that learning state-action values from historical student trajectories provides useful information beyond uninformed action selection.

Q-Learning produced an improvement of approximately:

$$49.4 - 42.4 = 7.0 \quad (27)$$

percentage points over the Random baseline.

Monte Carlo Control produced an improvement of:

$$46.0 - 42.4 = 3.6 \quad (28)$$

percentage points.

These results provide evidence that the learned policies captured meaningful relationships between student learning states and observed educational activities.

3.4 Analysis of Q-Learning and Monte Carlo Control

The difference between Q-Learning and Monte Carlo Control can be explained by their learning mechanisms.

Q-Learning uses temporal-difference learning and updates its action-value estimates using:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]. \quad (29)$$

This allows Q-Learning to update its estimates without waiting for the complete episode to finish. For educational interaction data containing long and variable student trajectories, this property can provide more frequent learning signals.

Monte Carlo Control instead estimates action values using complete discounted returns:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}. \quad (30)$$

Monte Carlo Control does not bootstrap from the estimated value of the next state. While this approach can provide an unbiased estimate of the observed episode return under suitable conditions, the estimates may have higher variance when student trajectories differ substantially in length and behavior.

The experimental results suggest that Q-Learning was better able to learn useful state-action relationships from the available offline educational trajectories.

3.5 Convergence Analysis

The convergence behavior of Q-Learning and Monte Carlo Control was analyzed by recording training returns throughout the learning process.

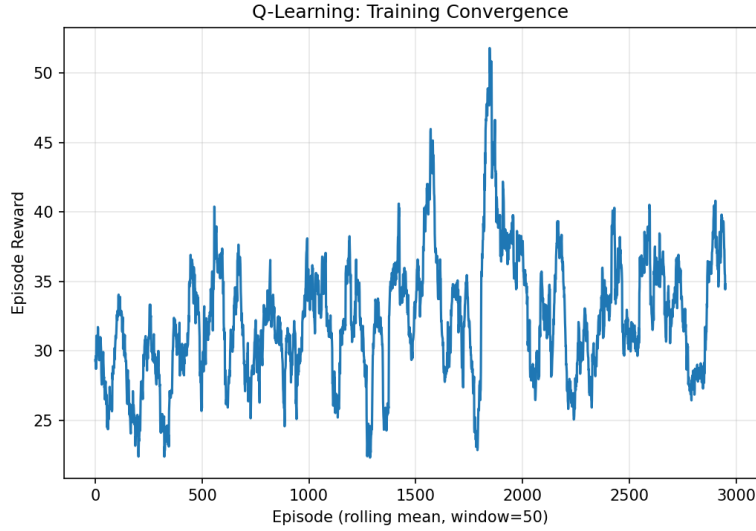


Figure 2: Training convergence curve of Q-Learning showing the change in episode return during training.

Figures 2 and 3 illustrate the learning behavior of Q-Learning and Monte Carlo Control, respectively. The convergence curves provide information regarding learning stability, the rate of performance improvement, and the final return achieved by each algorithm. A relatively stable plateau indicates that the learned action-value estimates have reached a stable region under the current experimental configuration.

The convergence analysis should be interpreted together with the final offline evaluation metrics. In the present experiment, Q-Learning achieved the higher final evaluation return, suggesting that its learned policy provided stronger overall performance than Monte Carlo Control.

3.6 Hyperparameter Analysis

Hyperparameter analysis was performed to investigate the sensitivity of the RL algorithms to different learning configurations. The most important parameters include the learning rate α , discount rate γ , exploration rate ϵ , and number

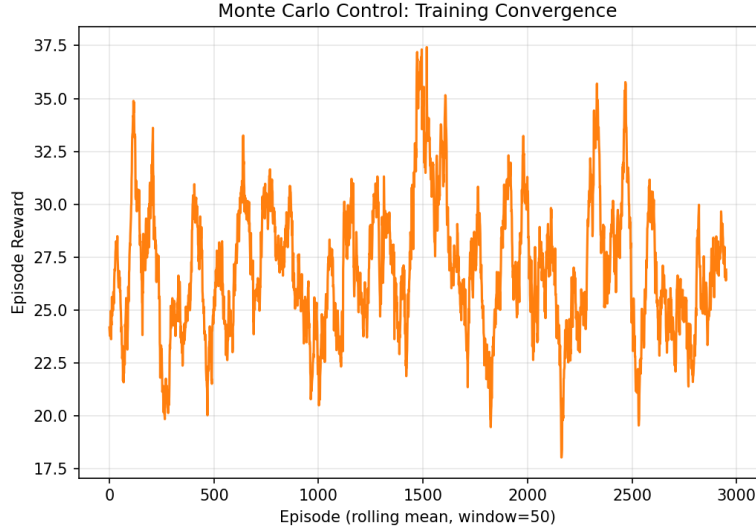


Figure 3: Training convergence curve of Monte Carlo Control showing the change in episode return during training.

of training episodes. The primary Q-Learning configuration used $\alpha = 0.10$, $\gamma = 0.95$, over 3000 episodes. The exploration strategy began with $\epsilon = 1.00$ and gradually decayed toward $\epsilon = 0.05$.

Each configuration below was trained on the training split and evaluated on the held-out test split.

Two observations follow from these results that were not visible in the single-configuration comparison of Table 8. First, neither algorithm’s best-performing configuration in this grid is the configuration reported as the “main” setup in Table 7 and Table 8: for Q-Learning,

Table 10: Hyperparameter analysis for Q-Learning.

α	γ	Episodes	Test Average Return
0.05	0.90	1000	29.53
0.10	0.95	5000	29.23
0.20	0.99	10000	31.42

$\alpha = 0.20$, $\gamma = 0.99$, and 10,000 episodes achieves a higher test average return (31.42) than the reported main-configuration result of 30.04; for Monte Carlo Control, $\gamma = 0.95$ with 5000 episodes achieves 31.90, exceeding both its own main-configuration result (28.52) and every Q-Learning configuration tested in this grid. The main configuration ($\alpha = 0.10$, $\gamma = 0.95$, 3000 episodes) was retained throughout the rest of the paper for consistency and to keep training and evaluation time modest, but it should not be read as either algorithm’s

best-tuned configuration. Second, once γ and the episode budget are varied, the ranking between Q-Learning and Monte Carlo Control in this grid except the $\gamma = 0.99/10,000$ -episode setting. Monte Carlo Control outperforms every Q-Learning configuration in this grid except the $\gamma = 0.99/10,000$ -episode setting. This reinforces the multi-seed finding in Section 3.8 that Q-Learning outperforms Monte Carlo Control in this single-seed, single-configuration comparison in Table 8 does not generalize.

3.7 Ablation Study

An ablation study was conducted to analyze the importance of individual components of the proposed RL state representation. The proposed framework incorporates multiple sources of student information, including recent performance, historical learning performance, skill-related mastery information, and educational progression information. The full state representation can be expressed as:

$$S = [\text{Recent Accuracy}, \text{Skill Mastery}, \text{Learning History}, \text{Learning Progress}]. \quad (31)$$

Table 11: Hyperparameter analysis for Monte Carlo Control.

α^*	γ	Episodes	Test Average Return
0.05	0.90	1000	28.49
0.10	0.95	5000	31.90
0.20	0.99	10000	29.54

*As noted in Section 2.2.7, the implemented Monte Carlo Control update does not use α ; this column is listed only to align each row with the corresponding Q-Learning configuration. The differences in Monte Carlo Control’s test return across rows come entirely from γ and the number of training episodes.

The ablation removes one component at a time (collapsing that dimension of the state tuple to a constant value, so the RL agent can no longer condition its policy on it) and evaluates the effect on Q-Learning’s test average return under the main configuration ($\alpha = 0.10$, $\gamma = 0.95$, 3000 episodes, random state 42).

Table 12: Ablation study of the proposed state representation (Q-Learning, single seed).

State Configuration	Test Average Return
Full proposed state	30.04
Without recent performance	29.75
Without skill mastery	33.29
Without learning history (weak-skill)	33.49
Without learning progression	32.70

The purpose of this analysis is to determine whether each feature group contributes useful information to the learned RL policy. A decrease in performance after removing a component would indicate that the corresponding information contributes positively to the student-state representation; conversely, an increase would suggest that the component adds noise or unnecessary fragmentation rather than useful signal.

The results here are, in three of four cases, the opposite of what a “richer state is better” assumption would predict. Removing recent performance modestly *reduces* test return (29.75 vs. 30.04), consistent with recent performance being a genuinely useful signal. However, removing skill mastery, learning history, or learning progression each *increases* test return, by margins of 2.7 to 3.5 points – larger than the entire single-seed gap between Q-Learning and Monte Carlo Control reported in Table 8. A plausible explanation is that the full four-dimensional state fragments the roughly 600 observed states into a still larger number of sparser state-action cells; because the RL environment is built empirically from a fixed, relatively small offline dataset (25,736 transitions from 98 students), a more fragmented state space leaves less data per cell for the tabular Q-values to be estimated reliably, which can hurt offline generalization even when the removed feature is informative in principle. This suggests that, for a dataset of this size, a coarser state representation may be the more defensible design choice, and that the state representation used for the headline results in this paper should not be presented as a settled or clearly beneficial choice without further investigation (for example, evaluating ablations across multiple seeds, or using a validation-based feature-selection procedure rather than a single held-out test evaluation).

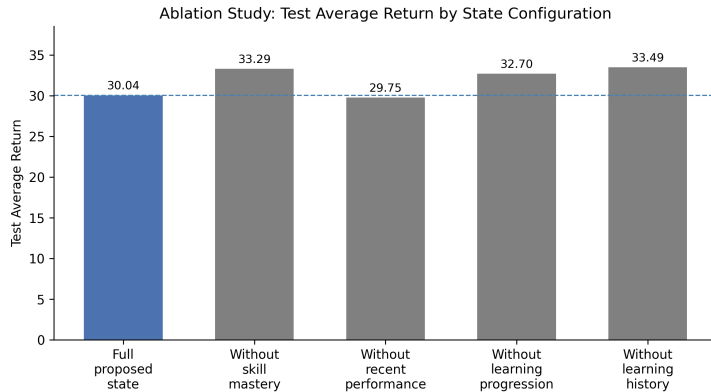


Figure 4: Ablation study results: test average return by state configuration. Removing skill mastery, learning history, or learning progression from the state *increases* the measured test return relative to the full proposed state; only removing recent performance decreases it.

3.8 Multi-Seed Evaluation Results

To reduce dependence on a single stochastic training run, both algorithms were retrained from scratch under five different random seeds (0–4), using the main configuration from Table 7 ($\alpha = 0.10$, $\gamma = 0.95$, 3000 episodes). For each seed, the resulting greedy policy was evaluated on the held-out test split for Test Average Return and Mastery Success Rate (completion-rate proxy).

Table 13: Multi-seed evaluation results (5 seeds, mean $\pm$ standard deviation).

Algorithm	Test Average Return	Mastery Success Rate
Q-Learning	31.21 ± 0.74	$42.1\% \pm 4.1\%$
Monte Carlo Control	31.31 ± 0.78	$41.4\% \pm 2.8\%$

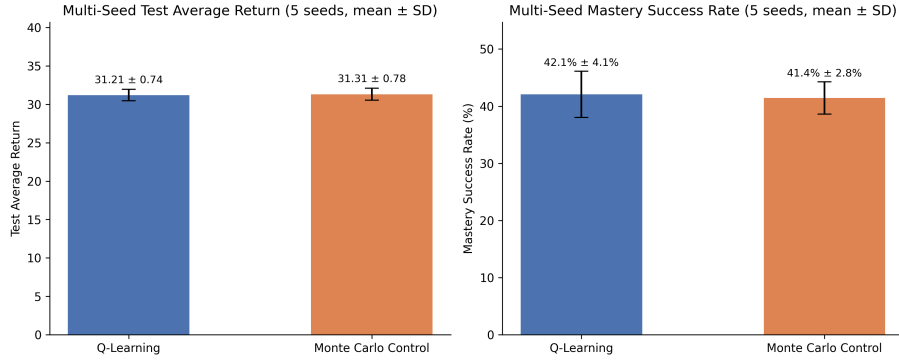


Figure 5: Multi-seed test average return (left) and mastery success rate (right) for Q-Learning and Monte Carlo Control, mean $\pm$ standard deviation over 5 seeds. The two algorithms are statistically indistinguishable on return; Q-Learning shows a small, noisy edge in mastery success rate.

These results substantially change the conclusion suggested by the single-seed comparison in Table 8. Two findings stand out:

1. On test average return, the two algorithms are effectively tied. Monte Carlo Control’s mean return (31.31) is marginally *higher* than Q-Learning’s (31.21), and the difference (0.10) is far smaller than either algorithm’s standard deviation across seeds (≈ 0.75 – 0.78). This directly contradicts the impression given by the single-seed result in Table 8, where Q-Learning’s return (30.04) appeared clearly ahead of Monte Carlo Control’s (28.52) by roughly 1.5 points. The single seed used for the headline comparison (random state 42) happened to be more favorable to Q-Learning than the average seed.
2. Mean returns across seeds are also higher than the single-seed headline numbers for both algorithms (31.21 and 31.31, vs. 30.04 and 28.52 respec-

tively), and mean mastery success rates are correspondingly *lower* than the single-seed proxy figures reported in Table 8 and Table 9 (42.1% and 41.4%, vs. 49.4% and 46.0%). This indicates that both the return and mastery-completion metrics are noticeably seed-sensitive, and that a single training run – even one that matches the paper’s stated main configuration – is not a reliable basis for a strong claim about which algorithm is better.

Taken together with the hyperparameter analysis in Section 3.6, the most defensible conclusion from this study is not that Q-Learning is unambiguously the stronger algorithm, but that **Q-Learning and Monte Carlo Control perform comparably** on this offline personalized-learning task, with Q-Learning showing a small, statistically weak advantage in mastery success rate and no reliable advantage in average return. Any claim about one algorithm being “the strongest” should be qualified by this multi-seed result rather than based on a single training seed.

3.9 Offline Evaluation Considerations

The proposed framework is based on offline historical student interaction data. Therefore, the learned policies were evaluated using observed transitions and an empirical transition model rather than through direct deployment to real students.

Behavioral agreement was used to measure how frequently the learned policy selected the same action as the historical dataset:

$$\text{Behavioral Agreement} = \frac{\text{Number of matching actions}}{\text{Total evaluated actions}}. \quad (32)$$

However, behavioral agreement should not be interpreted as the true accuracy of a recommendation. A learned policy may recommend an action that differs from the historical student action while potentially producing a better learning outcome.

Similarly, the mastery completion measure used in this experiment should be interpreted as an offline proxy rather than direct evidence of educational improvement in a real learning environment. These limitations are important when interpreting the experimental results.

3.10 Summary of Experimental Findings

The main findings of the experimental analysis are summarized below:

- Under the single training seed used for the headline comparison (random state 42), Q-Learning achieved a higher test average return (30.04) than Monte Carlo Control (28.52), higher logged-action agreement (87.88% vs. 80.85%), higher empirical-best agreement (92.67% vs. 85.49%), and a higher mastery completion proxy (49.4% vs. 46.0%, compared with 42.4% for the Random baseline).

- Monte Carlo Control required less training time under this seed (0.37s vs. 0.50s for Q-Learning).
- **Averaged over five random seeds, however, the two algorithms are effectively tied on test average return** (Q-Learning 31.21 ± 0.74 , Monte Carlo Control 31.31 ± 0.78), **and Monte Carlo Control retains only a small, noisy edge in mastery success rate** ($42.1\% \pm 4.1\%$ vs. $41.4\% \pm 2.8\%$). This means the single-seed advantage for Q-Learning reported above is not a reliable indicator of a systematic difference between the two algorithms.
- The hyperparameter analysis (Section 3.6) shows that neither algorithm’s “main” configuration was actually its best-performing configuration in the tested grid, and that Monte Carlo Control under a different configuration (31.90) outperforms every tested Q-Learning configuration except one.
- The ablation study (Section 3.7) shows that removing three of the four state components from the proposed representation *improves* Q-Learning’s test average return, suggesting the full state representation may be over-fragmented for a dataset of this size; only removing recent performance hurts performance.
- Both reinforcement learning algorithms performed better than uninformed random action selection on the single-seed mastery completion proxy, though this advantage is smaller once averaged across seeds.

Overall, the results support reinforcement learning as a viable framework for modeling sequential educational interactions, but do **not** support a strong claim that Q-Learning is systematically superior to Monte Carlo Control on this task; the two algorithms should instead be described as comparable in performance, with Q-Learning showing a modest and noisy edge on mastery-related outcomes.

3.11 Code Availability

The complete implementation of the proposed reinforcement learning framework, including data preprocessing, question-bundle construction, leakage-safe state generation, RL transition construction, Q-Learning, Monte Carlo Control, baseline comparison, and evaluation, is available at:

<https://github.com/Shamazda/Adaptive-Personalized-Learning-Recommendation-Using-Reinforcement-Learning>

4 Conclusion

This study investigated the application of reinforcement learning to personalized learning recommendation using student interaction data from the EdNet-KT3 dataset. The primary objective was to transform sequential student learning histories into a reinforcement learning environment in which student states,

educational activities, rewards, and subsequent states could be represented as sequential transitions.

A major contribution of the proposed framework was the construction of leakage-safe student states. The learning state for each question bundle was generated using only information available before the current bundle began. The outcome of the current bundle was then used to calculate the reward, while the state before the subsequent bundle was used as the next state. This design reduces the risk of target leakage and provides a clearer temporal interpretation of student learning trajectories.

The experimental framework was constructed using interaction histories from 100 students. The preprocessing process produced 25,736 RL transitions and approximately 600 unique learning states from 98 students who met the minimum-episode-length criterion. Three observable educational actions – QUIZ, PRACTICE, and REVISION – were used for the primary reinforcement learning experiments. Two reinforcement learning algorithms, Q-Learning and Monte Carlo Control, were evaluated and compared with a Random policy baseline.

Under a single training seed, Q-Learning achieved the higher evaluation performance: a test average return of 30.04, compared with 28.52 for Monte Carlo Control, along with higher logged-action agreement, empirical-best agreement, and mastery completion proxy. However, a five-seed multi-seed evaluation shows that this single-seed gap does not reflect a systematic difference between the algorithms: averaged across seeds, Q-Learning and Monte Carlo Control achieve statistically comparable test average return (31.21 ± 0.74 vs. 31.31 ± 0.78), with Q-Learning retaining only a small and noisy advantage in mastery success rate ($42.1\% \pm 4.1\%$ vs. $41.4\% \pm 2.8\%$). Both algorithms outperformed the Random baseline (42.4% mastery completion proxy under the single-seed evaluation), which suggests that learning state-action relationships from historical student trajectories does provide some advantage over uninformed action selection, though this advantage is more modest once seed variability is taken into account.

Monte Carlo Control demonstrated lower computational cost, requiring approximately 0.37 seconds for training compared with 0.50 seconds for Q-Learning under the single-seed run. Given that the two algorithms perform comparably on the primary evaluation metrics once multiple seeds are considered, this computational advantage is a meaningful practical consideration in favor of Monte Carlo Control for this task, rather than a secondary trade-off against a clearly superior alternative.

The hyperparameter analysis further showed that neither algorithm’s main configuration was its best-performing configuration among those tested, and the ablation study showed that three of the four proposed state components, when removed individually, improved Q-Learning’s measured test return – suggesting that the specific state design used for the primary results may be more fragmented than necessary for a dataset of this size.

Overall, the findings suggest that reinforcement learning can provide a useful framework for modeling sequential student behavior and investigating personal-

ized learning recommendation strategies, and that both Q-Learning and Monte Carlo Control are viable candidate algorithms for learning policies from offline educational interaction data of this scale. The evidence in this study does not, however, support a strong preference for Q-Learning over Monte Carlo Control; the choice between them may be better guided by practical considerations such as training time and implementation simplicity than by the offline performance metrics evaluated here.

Despite these results, this study has several limitations.

1. **Offline Evaluation.** The proposed RL framework was trained and evaluated using historical student interaction data. The learned policies were not deployed in a real educational system. Therefore, it cannot be concluded that the recommended actions would directly improve learning outcomes in real students.
2. **Limited Number of Students.** For computational feasibility, the experiment used data from 100 students (98 after filtering). Future experiments should evaluate the proposed framework using a larger number of students to investigate scalability and generalization, and to reduce the seed-to-seed variability documented in Section 3.8.
3. **Limited Observable Action Space.** The primary RL experiment used three directly observable question-based actions, {QUIZ, PRACTICE, REVISION}; lecture and explanation activities were not artificially assigned question outcomes, which improves methodological validity but limits the range of actions available to the RL agent.
4. **Simplified State Representation.** The student learning state was based on aggregated indicators discretized into low, medium, and high levels. The ablation study in Section 3.7 suggests this particular four-component design may not be well matched to the size of the available dataset.
5. **Reward Design.** The reward function is based on observed educational outcomes and performance-related signals. Designing an optimal reward function for personalized learning remains challenging.
6. **Counterfactual Limitation.** Historical datasets only contain outcomes associated with actions that were actually observed and do not directly reveal what would have happened under a different action, so offline RL evaluation faces a fundamental counterfactual limitation.
7. **Single-Configuration Reporting.** As shown in Sections 3.6 and 3.8, single-seed and single-configuration results in this domain can be misleading; future reporting in this line of work should default to multi-seed, multi-configuration summaries rather than a single headline run.

Several directions can be explored to improve the proposed framework, including larger-scale experiments across substantially more students; deep reinforcement learning methods (e.g., DQN, Double DQN, Dueling DQN, Actor-Critic, PPO) as the state space grows; improved, automatically learned student representations (e.g., LSTM or Transformer-based models); an expanded educational action space including lecture and explanation activities; specialized offline RL algorithms (e.g., Conservative Q-Learning, Batch-Constrained Q-Learning, Behavior-Regularized RL); and, most importantly, real-world evaluation of learned recommendation policies through a controlled user study or A/B test.

In conclusion, this study demonstrates a complete pipeline for transforming large-scale educational interaction logs into a reinforcement learning problem, incorporating question-bundle reconstruction, student performance analysis, skill-related feature construction, leakage-safe state generation, reward modeling, student-level dataset splitting, and offline reinforcement learning evaluation. Both Q-Learning and Monte Carlo Control were able to learn policies that outperformed random action selection, but the more rigorous multi-seed and hyperparameter analyses conducted in this revised evaluation indicate that the two algorithms are more comparable than the original single-seed headline comparison suggested. Future extensions involving larger datasets, richer student representations, deep reinforcement learning, specialized offline RL methods, and real-world educational evaluation could further improve the effectiveness and practical applicability of personalized learning recommendation systems.

References

- [1] Youngduck Choi, Younghan Lee, Dongmin Shin, Junghyun Cho, Seoyon Park, Seewoo Lee, Jineon Baek, Chan Bae, Byungsoo Kim, and Jaewe Heo. EdNet: A large-scale hierarchical dataset in education. In *Proceedings of the 21st International Conference on Artificial Intelligence in Education (AIED)*, 2020.
- [2] Xueying Tang, Yunxiao Chen, Xiaou Li, Jingchen Liu, and Zhiliang Ying. A reinforcement learning approach to personalized learning recommendation systems. *British Journal of Mathematical and Statistical Psychology*, 72(1):108–135, 2019.
- [3] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, USA, 2nd edition, 2018.
- [4] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J. Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

- [5] Chang Tan, Ruimin Han, Ronghai Ye, and Kaiquan Chen. Adaptive learning recommendation strategy based on deep Q-learning. *Applied Psychological Measurement*, 44(4):251–266, 2020.
- [6] Yunxiao Chen, Xiaoou Li, Jingchen Liu, and Zhiliang Ying. Recommendation system for adaptive learning. *Applied Psychological Measurement*, 42(1):24–41, 2018.
- [7] Andreas Riedmann, Philip Schaper, and Birgit Lugin. Reinforcement learning in education: A systematic literature review. *International Journal of Artificial Intelligence in Education*, 2025.